DEVMEET | ENGINEERING WALKTHROUGH

One Interview, Start to Finish

End-to-end backend flow with exact source line references and concise explanations

What this guide follows

Create interview - authenticate and provision session - start - stream AI questions - save transcript/code - monitor and pause/resume - complete - outbox - feedback report

Prepared for a technical interview walkthrough

Repository: HemanshuTala/devmeet-v2

Reference date: 21 October 2026

A presentation-friendly guide: each numbered step gives the concrete request/data flow, then cites the code that implements it. Optional or partially implemented behavior is identified rather than presented as guaranteed.

1. The whole flow at a glance

The Orchestrator owns session state and transcript storage. It does not generate the live AI response itself: the browser calls AI Interviewer directly, then saves turns back to the Orchestrator.

1 - SETUP Create-session form	2 - PROVISION Gateway to Orchestrator	3 - LIVE INTERVIEW AI stream + saved turns	4 - FINISH Complete + outbox + report
Chooses track, difficulty, duration, company/focus.	Auth + quota + active check + optional Search seed; writes session.	AI Interviewer returns SSE; Orchestrator stores candidate and interviewer turns.	Completion event dispatches transcript to Feedback Service.

Core services and responsibilities

Service	What it does in this flow
API Gateway - :8000	Routes session endpoints to Orchestrator and AI endpoints to AI Interviewer.
Orchestrator - :8003	Owns sessions, transcript turns, code snapshots, lifecycle state, completion event.
AI Interviewer - :8004	Authenticates user, generates streamed questions and optional hints/transcription.
Feedback - :8007	Evaluates transcript, saves scorecard, may render/store a PDF report.
PostgreSQL / Redis / S3	Durable records / coordination-cache / optional archival snapshots.

One important routing distinction
Session management: browser -> Gateway -> Orchestrator. **AI question generation:** browser -> Gateway -> AI Interviewer. The Orchestrator does not proxy the live AI stream.

Entry references: `api-gateway/nginx.conf:134-177` | `orchestrator-service/app/main.py:44-90` | `ai-interviewer-service/app/routes.py:375-552`

2. Setup and session creation

Candidate selects interview settings on the dashboard. The browser submits them through the centralized API client; the Gateway sends the request to the Orchestrator.

1 Candidate chooses interview settings

The setup page packages interview type, difficulty, duration, and optional company/focus data. On success it navigates to the new session page.

CODE: `frontend/src/app/dashboard/create-session/page.tsx:356-394`

2 Frontend calls the session API

React Query calls `sessionApi.create()`; the API client maps that mutation to `POST /api/v1/sessions`.

CODE: `frontend/src/hooks/queries/useSessions.ts:69-87` | `frontend/src/lib/api.ts:411-429`

3 Gateway routes and forwards the request

Nginx proxies `/api/v1/sessions` to `orchestrator-service:8003`, forwarding authorization, request ID, and idempotency headers.

CODE: `services/api-gateway/nginx.conf:134-153`

4 Orchestrator authenticates and validates

The endpoint uses an Auth Service-backed user dependency and Pydantic enums/limits. An optional idempotency key can replay a prior response.

CODE: `services/orchestrator-service/app/routers/deps.py:96-116` | `services/orchestrator-service/app/models.py:28-185` | `services/orchestrator-service/app/routers/sessions.py:70-91`

5 Checks and optional Search context run in parallel

Within a Redis user lock, Orchestrator checks/consumes User quota, checks for an existing active session, and requests a random Search item. Search metadata can fill missing focus/company fields.

CODE: `services/orchestrator-service/app/routers/sessions.py:92-143` | `services/orchestrator-service/app/clients.py:53-93,123-146` | `services/orchestrator-service/app/routers/deps.py:47-84`

6 Session is persisted and returned

A `created` row is inserted into PostgreSQL; Redis stores the active-session pointer, and the API returns the session ID and metadata.

CODE: `services/orchestrator-service/app/database.py:63-101` | `services/orchestrator-service/app/routers/sessions.py:144-183` | `services/orchestrator-service/app/redis_client.py:170-190`

What the candidate receives

The important result is the session UUID. The frontend navigates to `/interview/{id}`; the created session waits for the candidate to start. The Search seed does not itself execute the interview - AI Interviewer generates the live question from session metadata and transcript context.

3. Start the live interview and exchange turns

On opening the interview page, the frontend fetches the session. When the candidate starts, it begins the timer/heartbeat and starts AI question streaming while it confirms the backend state transition.

7

Load session state and prior transcript

The interview page fetches session details and, for an active/resumed session, loads saved turns to rebuild the chat. A fresh session stays at the start screen.

CODE: `frontend/src/app/interview/[id]/page.tsx:212-268` | `services/orchestrator-service/app/routers/sessions.py:215-255`

8

Start changes the durable state

The browser calls `POST /sessions/{id}/start`. Orchestrator checks ownership and allowed state, writes `in_progress`, and invalidates cache.

CODE: `frontend/src/app/interview/[id]/page.tsx:272-300` | `services/orchestrator-service/app/routers/sessions.py:257-314` | `services/orchestrator-service/app/models.py:47-84`

9

AI question request goes directly to AI Interviewer

The streaming hook sends session metadata/history and bearer token to `/api/v1/interview/question/stream`. The Gateway proxies to port 8004 with buffering disabled for SSE.

CODE: `frontend/src/hooks/interview/useAIStreaming.ts:80-145` | `services/api-gateway/nginx.conf:160-181` | `services/ai-interviewer-service/app/routes.py:375-552`

10

AI streams tokens; browser renders question

AI Interviewer validates the token through Auth Service, streams model output in SSE packets, and can fall back to scripted prompts. The hook progressively updates chat and may speak the final question with browser TTS.

CODE: `services/ai-interviewer-service/app/routes.py:84-133,375-552` | `frontend/src/hooks/interview/useAIStreaming.ts:145-240`

11

Both sides of the conversation are saved

Candidate input is saved to Orchestrator; after streaming, the AI question and optional hints/follow-ups are saved as an interviewer turn. PostgreSQL orders transcript records by turn number.

CODE: `frontend/src/app/interview/[id]/page.tsx:305-319` | `frontend/src/hooks/interview/useAIStreaming.ts:240-258` | `services/orchestrator-service/app/routers/turns.py:24-104` | `services/orchestrator-service/app/database.py:200-241`

Voice and code are adjacent paths

Speech input uses browser Web Speech when available; otherwise MediaRecorder sends audio to AI Interviewer transcription. During DSA tasks, code runs in the separate Code Execution Service and Orchestrator can store a code snapshot. These are optional paths around the main chat loop.

Voice: `frontend/src/hooks/interview/useSpeechRecognition.ts:1-200` | `services/ai-interviewer-service/app/routes.py:851-873` | Code: `frontend/src/app/interview/[id]/page.tsx:490-530` | `services/orchestrator-service/app/routers/proctoring.py:34-77`

4. Keep the session healthy and allow control actions

During the interview, the browser timer tracks elapsed time and sends periodic heartbeats. Pause/resume, cancellation, recording consent, code history, and proctoring counts are separate Orchestrator endpoints.

12

Timer and heartbeat keep state aligned

The frontend sends a heartbeat every 15 seconds. The backend records last activity, and the client reacts to automatic completion/cancellation responses.

CODE: `frontend/src/hooks/interview/useSessionTimer.ts:144-195 | services/orchestrator-service/app/routers/sessions.py:440-509 | services/orchestrator-service/app/database.py:337-351`

13

Pause and resume are lifecycle transitions

Pause stores elapsed time and `paused_at`. Resume verifies the session is within the 30-minute pause window and restores `in_progress`.

CODE: `frontend/src/app/interview/[id]/page.tsx:541-560 | services/orchestrator-service/app/routers/turns.py:106-204 | services/orchestrator-service/app/models.py:86-104`

14

Proctoring/code data are stored separately

The browser reports tab-switch/paste events; Orchestrator increments counters. Code submissions are saved as versioned rows. These endpoints store telemetry; the frontend also implements warning/termination UX.

CODE: `frontend/src/hooks/interview/useCheatingDetection.ts:61-104 | services/orchestrator-service/app/routers/proctoring.py:34-123 | services/orchestrator-service/app/database.py:243-335`

15

Cancel is terminal and clears active-session state

Candidate cancellation moves an unfinished session to `cancelled`, invalidates Redis cache, and clears the active-session pointer.

CODE: `frontend/src/app/interview/[id]/page.tsx:321-334 | services/orchestrator-service/app/routers/sessions.py:510-537 | services/orchestrator-service/app/routers/deps.py:86-94`

State-machine quick reference

From	Action	To	Code
created	Start	in_progress	sessions.py:257
in_progress	Pause	paused	turns.py:107
paused	Resume	in_progress	turns.py:164
created / in_progress / paused	Complete or cancel	completed / cancelled	sessions.py:316, 511

5. Completion, outbox, and final feedback

The final report is downstream work. Orchestrator first commits the completed state and event together; a background publisher later dispatches the transcript to Feedback Service.

16

Explicit completion updates state and writes outbox atomically

The completion endpoint verifies ownership/state and uses one PostgreSQL transaction for the session update plus `session.completed` event. If either operation fails, the transaction does not leave only one side committed.

CODE: `frontend/src/app/interview/[id]/page.tsx:337-377 | services/orchestrator-service/app/routers/sessions.py:315-438 | services/orchestrator-service/app/outbox.py:50-83`

17

Outbox worker reads and dispatches completion

The worker polls unpublished rows, fetches chronological turns, and POSTs session metadata/transcript to `/api/v1/feedback/generate`. A successful response marks the event published.

CODE: `services/orchestrator-service/app/main.py:44-62 | services/orchestrator-service/app/outbox.py:86-188,190-268`

18

Feedback Service evaluates and persists report

Feedback Service applies a participation guard, evaluates with Groq when configured (otherwise a local heuristic), calculates a static percentile, optionally renders/uploads a PDF, and saves the report to PostgreSQL.

CODE: `services/feedback-service/app/routes.py:41-116 | services/feedback-service/app/generator.py:120-195 | services/feedback-service/app/database.py:76-153`

19

Feedback page loads finished report

The browser retrieves session/transcript and first requests saved feedback. If missing, it can request generation itself; on API failure it computes a browser-side fallback.

CODE: `frontend/src/app/interview/[id]/feedback/page.tsx:109-190 | frontend/src/lib/api.ts:567-572 | services/feedback-service/app/routes.py:132-171`

20

Optional S3 archive preserves replay data

After explicit completion, a background task can package session metadata, transcript, and code history into encrypted S3 JSON; it records the object key in PostgreSQL.

CODE: `services/orchestrator-service/app/routers/sessions.py:405-420 | services/orchestrator-service/app/routers/snapshots.py:17-37 | services/orchestrator-service/app/s3_service.py:100-163`

Important delivery detail
This repository uses a PostgreSQL outbox plus HTTP to Feedback Service; the shown worker does not publish to RabbitMQ. Delivery is retryable/at-least-once, so a request may repeat if downstream accepted it but the publisher did not record acknowledgment.

The feedback page also has a generate-if-missing path. If it races the outbox request, two evaluations may happen; the feedback table upsert prevents duplicate rows but not duplicate work.

6. What to say - and implementation caveats

A concise explanation for the interviewer

"The dashboard sends selected interview settings through the API Gateway to the Orchestrator. The Orchestrator authenticates the user, checks quota and active-session rules, and persists a session. During the interview, the browser calls AI Interviewer directly for SSE question generation and sends candidate/AI turns back to Orchestrator for PostgreSQL storage. Heartbeats and lifecycle endpoints keep session state synchronized. On explicit completion, Orchestrator atomically marks the session completed and inserts an outbox event; a worker reads the transcript and calls Feedback Service, which evaluates and saves the report. S3 snapshot archival is optional."

Know these accurate caveats

• **AI is a separate hop:** Orchestrator manages session; AI Interviewer generates live questions directly for the browser.

• **Outbox transport:** this implementation sends HTTP to Feedback Service, not RabbitMQ.

• **Fallback generation:** the feedback page can independently call generate when no saved report exists.

• **Heartbeat auto-completion:** the current heartbeat branch updates status but does not emit the same completion outbox event as explicit completion.

• **Optional integrations:** Search seed and S3 archival can be unavailable without blocking all session features.

• **Data ownership:** PostgreSQL is durable session/transcript storage; Redis is coordination/cache, not the record of truth.

Fast source index

Stage	Start line(s) to open in VS Code
Setup form and create	frontend/src/app/dashboard/create-session/page.tsx:356; sessions.py:70
Auth / quota / active / seed	routers/deps.py:47,96; clients.py:53,96,123
Session DB + state machine	database.py:63; models.py:28,47,70
Live interview stream	frontend/hooks/useAIStreaming.ts:80; AI routes.py:376; nginx.conf:160
Transcript turns	routers/turns.py:25,63; database.py:200,227
Heartbeat/pause/proctoring	sessions.py:441; turns.py:107,164; proctoring.py:35,80
Completion + report	sessions.py:316; outbox.py:190; feedback routes.py:42
Report page / archive	frontend feedback/page.tsx:109; snapshots.py:17

Line references match the repository snapshot used to prepare this walkthrough. In VS Code, use Ctrl+G and enter the cited line number.